

bespokelabsai / nimble

Q

+ ▾

<> Code Issues 2 Pull requests 2 Agents Actions Projects Security and quality Insights

Watch 8 ▾

Fork 153 ▾

☆

Local typed decisions, contrastive data curation, and model evaluation.

☆ 1.9k stars Fork 153 forks 8 watching Branches 0 tags Activity Custom properties

Public repository

m...

Branches

Tags

Go to file

Go to file

Add file

Code

.agents/skills/typesafe-ai

assets

data

deploy

docs

examples

nimble

requirements

tests

.gitignore

AGENTS.md

README.md

skills-lock.json

Bespoke Nimble

Data, Model, Recipe for an open Jev

[Model](#) · [Updates](#) · [Capabilities](#) · [Quickstart](#) · [Methodology](#) · [Documentation and development](#) · [Citation](#)

 [Introducing Bespoke Nimble. Serving reads the prompt once and then scores one answer token per question. Data curation changes one fact so that the correct answer flips. Training fine-tunes Qwen3.5-9B with LoRA on the answer tokens only. On 324 held-out examples, Bespoke-Nimble-9B matches 90.1% of the reference labels, compared with 66.4% for its base model and 93.2% for Jev 1.13.0.](#)

Nimble takes some text and a schema, and makes typed decisions about the text. The schema is the list of questions to answer. Each question is either a choice from a list that you give or a true or false question. For each question, Nimble returns the answer it picked and the probability of each allowed answer.

Nimble makes each decision in one step and does not write out any reasoning first, so it is fast (blazing fast!). Nimble is inspired by the System One approach of [TypeSafe's Jev](#). In this repository, we share our recipe for training such a model.

Note that we did not distill from Jev. The point of the repository is to show how to curate data, how to train, and to serve such a model, and encourage more research!

You can run [Bespoke-Nimble-9B](#) on a Mac with Apple Silicon or on a machine with an NVIDIA GPU.

Updates

- September 24, 2026: [Bespoke-Nimble-9B](#) now contains the latest checkpoint, with an 8,192-token context and up to 255 choices per field. Its default is $T=1.0$; the original release is preserved under the `original-2676` tag, and the separate v2 repository is unchanged. Update this checkout before loading the new release.
- 22 сентября 2026 года мы подогнали температуру для Bespoke-Nimble-9B. При такой температуре вероятности лучше соответствуют частоте правильных ответов. Модель выбирает те же ответы, что и раньше. Нулевые вероятности и значения Score меняются, поэтому, если вы сравниваете их с пороговым значением, проверьте его еще раз. См. [Температура вероятностей](#) и [PR #7](#).
- 20 сентября 2026 года мы опубликовали 2676 обучающих примеров и 324 контрольных примера для Bespoke-Nimble-9B. Мы по ошибке не включили их в первый релиз. См. [руководство по набору данных](#) и [PR #5](#).
- 19 сентября 2026 года мы увеличили лимит подсказок в хостинговом API до 8192 токенов для каждого вопроса. Модель обучалась на подсказках длиной до 2048 токенов, поэтому более короткие подсказки тестируются лучше. См. [руководство по развертыванию SGLang](#) и [PR №4](#).
- 18 сентября 2026 года [Эдгар Дайк](#) добавил общедоступный набор бенчмарков. С его помощью вы можете запустить Bespoke-Nimble-9B и Jev на тех же записях из 13 общедоступных подмножеств с разметкой, выполненной людьми. В [руководстве по общедоступным бенчмаркам](#) описаны шаги и результаты. См. [PR #2](#).

Возможности

Мы создали Nimble за один день, так что не удивляйтесь, если в нем будут шероховатости. Возможности Nimble обусловлены двумя факторами: во-первых, это базовая модель Qwen3.5-9B, во-вторых, наши обучающие данные, которые мы собрали для нескольких конкретных областей.

Что вы можете построить

Задача	Вы определяете	Ты возвращаешься
Перенаправление запроса	Куда можно поехать и когда это возможно	Выбранное направление и вероятность попадания в каждое из них
Проверьте условие	Вопрос, требующий ответа «да» или «нет», и доказательства	Правда или ложь, и вероятность каждого варианта
Применять политику	Правила и допустимые результаты	Типовое решение на основе предоставленного вами текста
Оцените результат	Упорядоченные уровни, каждый из которых имеет четкие критерии	Выбранный уровень и вероятность каждого уровня

Вы предоставляете контекст — текст, который нужно оценить, — и схему. Схема должна быть плоской, то есть не содержать вложенных полей. Каждое поле представляет собой перечисление или логическое значение. Поле перечисления имеет фиксированный список строковых вариантов, а логическое поле может принимать значения `true` или `false`.

У каждого допустимого ответа есть код длиной в один токен. Оценщик считывает логиты модели для этих кодов. Логиты — это необработанные оценки, которые модель присваивает каждому токenu. Оценщик преобразует логиты в вероятности с помощью функции `softmax`. Затем наш код на Python формирует выходные данные на основе этих вероятностей, поэтому нет необходимости сгенерированном JSON, который нужно разбирать. Если поле представляет собой упорядоченную рейтинговую шкалу, ваше приложение может использовать вероятности для расчета ожидаемого уровня.

На Mac `ParallelScorer` обрабатывает общий контекст один раз, а затем оценивает все поля параллельно. Модуль оценки CUDA оценивает каждое поле отдельно, каждый раз используя полную подсказку. Оба модуля оценки возвращают типизированный вывод. Они также возвращают логиты и вероятности ответов-кандидатов. Каждое поле оценивается отдельно, поэтому одно поле не может видеть ответ другого поля.

Что нельзя собрать с помощью текущей версии

- Nimble принимает только текст. Его нельзя использовать для оценки других типов входных данных, например изображений. Это так, даже несмотря на то, что базовая модель включает в себя визуальную составляющую.
- Nimble выбирает только из предложенных вами вариантов. Он не может писать текст самостоятельно, например объяснение. Он также не может возвращать вложенный JSON или фрагмент текста, взятый из контекста. В последней версии поле `enum` может содержать от 1 до 255 строковых вариантов, а поле `boolean` — два варианта.
- Вероятность не гарантирует, что ответ правильный. Nimble масштабирует их таким образом, чтобы сумма по всем предоставленным вами ответам равнялась 1. В последней контрольной точке используется $T=1.0$, и для нее не подбиралась

отдельная температура. В более ранних версиях были свои настройки температуры. См. [Температура вероятности](#). Вероятност 0,9 все равно не означает, что ответ будет правильным в 90 % случаев на ваших данных. Если ни один из ваших ответов не подходит, добавьте вариант «не соответствует». Прежде чем полагаться на какой-либо порог вероятности, протестируйте его на собственных данных.

- В последней версии каждое подмножество может содержать не более 8192 токенов. Это ограничение распространяется на схему и часть подмножества, в которой указывается поле для оценки. Nimble отклоняет более длинные подмножества. Поля могут зависеть друг от друга, поэтому ваш код должен проверять согласованность ответов для разных полей.

Эффективность Nimble зависит от тщательно отобранных данных и областей, представленных в обучающей выборке.

Протестируйте его на своих задачах. Но мы видим, что в новых областях Nimble в целом работает лучше, чем его базовая модель Qwen3.5-9B.

🔗 Быстрый Старт

Клонируйте репозиторий и перейдите в его папку. Выполните все приведенные ниже команды из этой папки.

```
git clone https://github.com/bespokelabsai/nimble.git nimble
cd nimble
```

Используйте Python 3.12. Чтобы запустить модель на Mac, вам понадобится Apple Silicon. Python также должен работать непосредственно в macOS, чтобы использовать Metal — интерфейс Apple для графического процессора. Чтобы запустить модель в Linux, вам понадобится графический процессор NVIDIA, поддерживающий BF16 — 16-битный формат чисел.

Без квантования одни только веса 9B занимают около 18 ГБ. Квантование означает хранение весов с меньшим количеством битов для экономии памяти. Во время работы модели требуется больше памяти. Приведенный ниже этап объединения выполняется на центральном процессоре. Для него нужна дополнительная оперативная память и место на диске как для базовых, так и для объединенных весов. На Mac с 64 ГБ памяти для этого доступно больше свободного места, чем на компьютере с 24 ГБ.

🔗 Скачать модель

Создайте среду Python для подготовки модели. В Linux эту среду также можно использовать для запуска модели на графическом процессоре. Для этого установите сборку PyTorch с поддержкой CUDA, которая работает с вашим драйвером графического процессора.

```
python3.12 -m venv .cache/venvs/nimble
source .cache/venvs/nimble/bin/activate
python -m pip install torch==2.8.0 -r requirements/training.txt
```

Следующий код принимает либо полную контрольную точку, либо адаптер PEFT LoRA. Для адаптера он загружает закрепленную базу и объединяет обученные веса. Он записывает разрешенную версию и путь к локальной модели для обоих примеров платформы. Для локального логического вывода не требуется ни TypeSafe, ни ключ API генерации.

```
питон - <<'Питон'
импорт hashlib
импорт данных JSON
из pathlib импорт путь

от huggingface_hub импорт snapshot_download

РЕПО = "bespokelabs/на заказ-шустрые-9B" # или "bespokelabs/на заказ-шустрые-9B-V2-это"
моментальный снимок = путь(snapshot_download(РЕПО, cache_dir=".кэш/huggingface/концентратор"))
contract_file = моментального снимка / "schema_config.формат JSON"
договор = формат JSON.нагрузок(contract_file.read_text()) если contract_file.существует() еще {}
если контракт:
    от трансформаторов импорт AutoTokenizer
    от шустрые.обучение.candidate_schema импорт validate_contract
    validate_contract(договор, AutoTokenizer.from_pretrained(моментальных снимков))

model_path = снимка
, если (кадр / "adapter_config.формат JSON").существует():
    импорт Факел
    из peft импорт PeftModel
    от трансформаторов импорт AutoTokenizer, Qwen3_5ForConditionalGeneration

    # адаптер релиз должен включать его прижали базы и оперативное договора.
    основание = Qwen3_5ForConditionalGeneration.from_pretrained(
        контракт["модель"], пересмотру=договором["пересмотр"],
        dtype=Факела.bfloat16, device_map="процессор",
    )
    адаптер = PeftModel.from_pretrained(базовая, снимок)
```

```

объединены = адаптера.merge_and_unload(safe_merge=истина)
model_path = путь(".кэш/модели") / ("шустрые-96-" + моментальных снимков.наименование)
merged.save_pretrained(model_path)
(model_path / "schema_config.json").write_text(json.dumps(contract, indent=2))
AutoTokenizer.from_pretrained(snapshot).save_pretrained(model_path)
# Сохранить идентификатор адаптера для автоматического выбора температуры после слияния.
(model_path / "READY.json").write_text(json.dumps({
    "модель": репозиторий, "редакция": snapshot.name ,
    "adapter_sha256": hashlib.sha256(
(снимок / "adapter_model.safetensors").read_bytes()
).hexdigest(),
}, отступ=2))

config = {
    "model_path": str(model_path.resolve()),
    "model_id": пено,
    "ревизия": snapshot.name ,
    "max_input_tokens": contract.get("max_length", 2048),
}
Путь(".cache/nimble-model.json").write_text(json.dumps(config, indent=2))
print("Ready:", model_path)
PYTHON

```

🔗 Mac с процессором Apple Silicon (MLX)

После этапа подготовки модели используйте отдельную среду MLX:

```

deactivate
python3.12 -m venv .venv-mlx
source .venv-mlx/bin/activate
python -m pip install -r requirements/mlx.txt

```

В Python передайте сохраненные настройки модели в `ParallelScorer`, чтобы она загрузила подготовленные веса 9B.

```

nimble from
Path import pathlib from
json import.scoring.parallel_scorer import ParallelScorer

config = json.loads(Path(".cache/nimble-model.json").read_text())
scorer = ParallelScorer(**config)

```

📌 Примечание

Если вызвать `ParallelScorer()` без аргументов, будет загружена модель Qwen3.5-4B, которую мы использовали в качестве базовой, а не Nimble. Средство запуска MLX не может напрямую загрузить папку с адаптером LoRA, поэтому используйте объединенную папку, которую вы подготовили выше. Средство запуска MLX также не поддерживает квантованные веса.

🔗 Linux с графическим процессором NVIDIA (CUDA)

Активируйте среду, в которой вы готовили модель. Затем проверьте, может ли PyTorch использовать графический процессор и поддерживает ли графический процессор BF16.

```

source .cache/venvs/nimble/bin/activate
python -c 'import torch; assert torch.cuda.is_available() and torch.cuda.is_bf16_supported()'

```

В Python загрузите те же подготовленные веса.

```

nimble from
Path import pathlib from
json import.scoring.cuda_scorer import CudaCandidateScorer

config = json.loads(Path(".cache/nimble-model.json").read_text())
scorer = CudaCandidateScorer(**config)

```

Вы также можете запустить адаптер без объединения, как и при оценке обучения. О том, как это сделать, читайте в [руководстве по обучению](#). Средство запуска CUDA оценивает каждое поле с помощью полного запроса. Таким образом, оно повторно обрабатывает общий контекст для каждого поля, в то время как средство запуска MLX обрабатывает его только один раз.

🔗 Примите типизированное решение

После загрузки любого из scorer продолжите работу в том же сеансе Python.

```
= schema {
  "priority": {
    "type": "enum",
    "choices": ["HIGH", "LOW"],
    "description": "Срочность в зависимости от текущего влияния на бизнес.",
    "choice_descriptions": {
      "HIGH": "Критически важная бизнес-операция в настоящее время заблокирована.",
      "LOW": "Необязательное усовершенствование, не оказывающее влияния на текущую деятельность.",
    },
  },
  "requires_review": {
    "type": "boolean",
    "description": "Указывают, не могут ли клиенты совершить покупку.",
  },
}
result = scorer.score("Платежный сервис недоступен для всех клиентов.", schema)
print(result["output"])
print(result["fields"]["priority"]["scores"]) # Вероятности кандидатов.
print(result["fields"]["priority"]["logits"])
```

Результат имеет такую форму. Значения зависят от модели:

```
{"priority": "HIGH", "requires_review": true}
```

Загрузите оценщик один раз и используйте его для каждого нового контекста. В последней контрольной точке используется $T=1,0$; следующая подобранная температура применима только к исходному релизу. Для версии 93ec5d6ff1a9cd31d6cc0e0c58d312465d36de7c Bespoke-Nimble-9B оба оценщика по умолчанию используют температуру 2,179. Оценщикам нужен полный хэш версии, а короткий хэш, например 93ec5d6, не подходит. На этапе загрузки выше полный хэш сохраняется в `.cache/nimble-model.json`. Мы подобрали эту температуру таким образом, чтобы вероятности лучше соответствовали частоте правильных ответов. См. [Температура вероятности](#). Для других моделей температура по умолчанию составляет 1.0, за исключением [Bespoke-Nimble-9B-v2](#): в версии 2 автоматически используется **2.179078721266035** в обоих оценщиках, включая локальные слияния, для которых `READY.json` определяет адаптер. Это стандартное значение, перенесенное из предыдущей версии, а не температура, подобранная отдельно для версии 2. В версиях Hub, предназначенных только для документации, сохраняется это значение по умолчанию. Для намеренного использования необработанных вероятностей версии 2 требуются `temperature=1.0` и `allow_uncalibrated=True`; при передаче только `temperature=1.0` возникает ошибка. Другие положительные температуры остаются явными переопределениями. См. [руководство по оценке](#) для получения информации о правилах схемы и способах запуска оценщика.

🔗 Методология

В этом разделе представлены данные и рецепт исходного релиза. Последняя контрольная точка указана в разделе [Обновления](#).

Методология подачи и обработки обучающих данных во многом основаны на [Bespoke-MiniCheck](#).

🔗 Служение

Мы следуем подходу, предложенному [Нильсом Рогге](#): см. этот [пост о том, как Jev выполняет декодирование](#).

В двух словах:

- Обрабатываем контекст и схему один раз (предварительно заполняем KV-кэш)
- Мы получаем оценки для интересующих нас токенов.

Мы использовали этот подход (нам не нужно было предварительно заполнять kv-cache) в Bespoke-MiniCheck, поскольку он всегда возвращал один кортеж json: `{"is_claim_supported_by_context": p}`.

🔗 Сопоставительный анализ данных

Проблема в том, что у нас нет доступа к вероятностям, полученным с помощью обучающей модели (или людей). Но, как вы видели выше, нам просто нужно каким-то образом получить логиты и убедиться, что они максимально точно отражают вероятности.

Мы подталкиваем модель к калибровке, чтобы она лучше принимала решения, создавая отрицательные примеры, которые заставляют модель лучше распознавать различия.

Поэтому мы подготовили обучающие данные с помощью нового метода, который мы назвали контрастным курированием данных. В рамках этого метода мы создаем два почти идентичных примера. Они отличаются одним важным фактом, и это отличие меняет правильный ответ. Все остальное остается неизменным, включая вопрос и политику. На основе этих пар модель учится определять, какие доказательства должны изменить ее решение.

 [Посмотрите видео о контрастном подборе данных: если заменить имя Мира на Ноа, ответ изменится с «верно» на «неверно».](#)

[Посмотрите 34-секундное объяснение](#) (без звука).

В этом примере правило таково: возврат средств разрешен только в том случае, если соответствующее разрешение подписал человек, который может одобрять возврат средств для этого аккаунта. В примере есть два предложения с доказательствами, и ответ должен быть дан на оба.

Доказательства	Первый пример	Измененный пример
Кто может одобрить возврат средств	Только Мира может разрешить возврат средств для аккаунта 42.	Неизменный
Запись авторизации	Единственное разрешение на возврат средств на счет 42 было подписано Мирой .	Единственное разрешение на возврат средств на счет 42 было подписано Ноем .
Разрешен ли возврат средств?	true	false

Наш конвейер курирования использует эту идею для задач Choice, Noul и Score. Он состоит из четырех этапов:

1. Проверяем правила принятия решений. Мы начинаем со схемы и политики обучающего источника. Мы записываем основные факты, от которых зависят правила. Затем мы проверяем, что правила могут приводить к разным ответам.
2. Создаем пару. Мы записываем два предложения с доказательствами, и для получения ответа нужны оба. Затем мы меняем не более восьми слов в одном из предложений. Мы вносим это изменение, чтобы изменить один факт, который мы называем ключевым, и тогда меняется и метка. Остальной контекст и другие факты остаются прежними.
3. Проверьте оба примера. Мы используем отдельные вызовы модели для проверки фактов в каждом примере. Мы также используем эти вызовы, чтобы убедиться, что пример соответствует правилам и что текст не намекает на ответ. Затем мы по очереди удаляем каждое предложение с доказательством. После удаления любого из этих предложений ключевой факт должен стать неизвестным, даже если весь остальной текст остается на месте. Таким образом, мы можем быть уверены, что никакой другой текст не раскрывает ответ.
4. Создаем метки. Мы используем код для применения проверенных правил к проверенным фактам. Мы сохраняем пару только том случае, если все необходимые проверки пройдены и две метки отличаются. Мы сохраняем все запросы и ответы модели, также результаты проверок. Вы можете использовать их для воспроизведения всего процесса в автономном режиме.

Примеры, из которых удалено предложение с доказательством, служат только для проверки. Мы не добавляем их в обучающие данные с метками, поскольку отсутствие доказательства не означает, что ответ неверный или что оценка низкая. Семейство источников — это группа примеров, созданная на основе одного обучающего источника. Мы оставляем оба примера из пары и все примеры из одного семейства источников в одном разбиении. Разбиение может быть обучающим или оценочным.

Текущий набор данных

Есть один обучающий набор данных и один проверочный.

Файл	Примеры	Использование
data/train.jsonl	2,676	Оригинальный обучающий набор для выпуска
data/eval.jsonl	324	Только замороженная окончательная оценка

Опубликованные обучающие данные модели охватывают **10 предметных категорий**. В таблицах ниже указано количество только обучающих примеров — 2676, а также количество примеров, не вошедших в обучающую выборку, — 324. Количество категорий указано в поле `domain` каждой записи; в выборку не вошли шесть из этих категорий.

Категория	Обучающие примеры	Приведенные примеры
Коммерция	242	58
Образование	230	70
Главная	300	0
Медиафайлы	256	44
Государственные услуги	194	106
Наука	300	0

Категория	Обучающие примеры	Приведенные примеры
Программное обеспечение	300	0
Цепочка поставок	270	30
Путешествия	284	16
Рабочее место	300	0
Итого	2,676	324

В каждом из этих примеров задается один из трех [типичных вопросов](#):

Тип задачи	Суждение	Обучающие примеры	Приведенные примеры
Выбор	Выберите одного кандидата	856	146
Noui (логическое значение)	Определите, выполняется ли условие	888	114
Оценка	Оцените уровень упорядоченности рубрики	932	64
Итого		2,676	324

Все метки являются синтетическими: их проверила модель, и ни один человек их не просматривал. При отдельных вызовах одной той же модели может возникать одна и та же ошибка, поэтому при проверке некоторые ошибки могут быть пропущены. Полный метод описан в [руководстве по генерации и воспроизведению](#).

Приведенная ниже команда проверяет файлы с данными. Для этого не нужен графический процессор, и команда не обращается к API. Она подтверждает, что файлы соответствуют сохраненным контрольным суммам и содержат ожидаемое количество примеров. Кроме того, она подтверждает, что ни один пример или семейство источников не присутствуют одновременно в обучающем и оценочном наборах.

```
.venv-curator/bin/python -m nimble.training.verify_dataset
```

См. [подробности о наборе данных](#) и [руководство по обучению](#). В руководствах по курированию также описаны более ранние способы создания данных. Мы больше не храним исходные папки, созданные с помощью этих методов.

 Точная Настройка

Тренер применяет LoRA к Qwen3.5-9B, оптимизируя кросс-энтропию по допустимым логитам кандидатов. Он напрямую обучается и типизированному решению. Сохраненные вероятности Jev доступны для последующей дистилляции мягких целей, если вам это нужно, но текущая цель обучения использует жесткие эталонные метки, которые не получены с помощью Jev.

В итоговом рецепте используются следующие настройки:

- LoRA, ранг 16
- Скорость обучения 5e-5
- Эффективный размер пакета 8
- Случайное начальное число 17
- Одна эпоха обучения

Он сохраняет линейную схему изменения скорости обучения на протяжении трех эпох, используемую при отборе, и останавливает после выбранной эпохи. При обучении используется алгоритм BF16 и ограничение на количество токенов в подсказке — 2048. Настройка проводилась на L40S, а финальная подгонка и оценка — на H100.

 Оценка на 324 контрольных примерах

 [Согласованность эталонных меток для тех же 324 примеров: Jev — 93,21 %, Bespoke-Nimble-9B — 90,12 %, Qwen3.8-27B — 84,88 %, Qwen3.5-9B — 66,36 %, Qwen3.5-4B — 61,42 %, Qwen3.5-0.8B — 45,37 %, Gemma 3 270M IT — 28,70 %.](#)

Модель	Эталонные соответствия	Соглашение
Gemma 3 270M IT	93/324	28.70%
Qwen3.5-0.8B	147/324	45.37%
Qwen3.5-4B	199/324	61.42%
Qwen3.5-9B	215/324	66.36%
Qwen3.8-27B	275/324	84.88%

Модель	Эталонные соответствия	Соглашение
Bespoke-Nimble-9B	292/324	90.12%
Jev 1.13.0	302/324	93.21%

На этих 324 примерах Bespoke-Nimble-9B сопоставила с эталонными метками на 17 больше, чем не дообученная модель 27B, что на 5,25 процентных пункта больше. Jev сопоставила с эталонными метками на 10 больше, чем Bespoke-Nimble-9B, что на 3,09 процентных пункта больше. Не дообученные модели — это модели, которые мы не дообучали для решения этой задачи. Мы протестировали все семь моделей на одних и тех же примерах с одними и теми же эталонными метками. Мы запустили Gemma и до конца настроенную модель Qwen на графическом процессоре H100. Для Bespoke-Nimble-9B мы использовали проверенные результаты предыдущего запуска на H100. Для Jev мы использовали результаты предыдущего запуска через его API.

Эталонные метки являются синтетическими. 324 примера образуют 162 пары тесно связанных между собой примеров. Все они происходят всего из шести семейств, так что это довольно узкий тест.

Для неадаптированных моделей мы вычисляли баллы для возможных ответов в формате FP32, 32-битном числовом формате. Для Bespoke-Nimble-9B мы сохранили уже проверенную настройку, при которой выходной слой вычисляется в формате BF16. В новом запуске неадаптированной модели 9B один из ответов, который ранее был равновероятным, перестал быть таковым. Это изменило количество ответов модели с 214 до 215. Для задач ранжирования мы засчитываем совпадение, когда наиболее вероятный уровень равен эталонному. В [сравниваемом файле](#) указаны приведенные выше значения совпадения с эталоном.

Мы проверили сохраненный адаптер двумя способами. После перезагрузки он выдавал точно такие же logits, как и раньше. Когда мы отключили адаптер, модель выдавала те же результаты, что и базовая модель. Мы проводили эти проверки только на пути CUDA, который загружает адаптер без его объединения с основной моделью. Мы не проводили отдельный тест качества для объединенной модели, который используется в руководствах по началу работы для Mac и Linux. Команды и файл контракта, сохраняемый с каждой моделью, см. в [руководстве по обучению](#). В файле контракта мы указываем базовую модель и формат подсказок, которые ожидает модель.

Внешние тесты с разметкой человеком для задач, выходящих за рамки этих обучающих категорий, можно найти в [руководстве по общедоступным бенчмаркам](#), где Bespoke-Nimble-9B и Jev тестируются на одних и тех же записях из тринадцати общедоступных подмножеств, начиная с VitaminC.

Вероятностная температура

Модель преобразует logits в вероятности с помощью функции softmax. Перед применением softmax logits делятся на число, которое называется температурой. При температуре выше 1 вероятности менее экстремальны. Порядок ответов по вероятности одинаков при любой температуре. Таким образом, модель выбирает один и тот же ответ при любой температуре. Сами вероятности меняются, как и ожидаемый уровень, который вы рассчитываете на их основе. Если ваш код сравнивает вероятность или ожидаемый уровень с пороговым значением, проверьте его еще раз.

В последней контрольной точке используется T=1,0 без отдельной подгонки. Приведенные ниже результаты относятся к исходному релизу. Мы выбрали одну температуру для версии 93ec5d6 Bespoke-Nimble-9B. Мы использовали два набора по 300 примеров, причем эти наборы были взяты из разных источников. Ни один из этих примеров не входит в обучающие данные или в 324 контрольных примера, описанных выше. В первом наборе мы выбрали температуру с наименьшей логарифмической потерей. Логарифмическая потеря — это средний отрицательный логарифм вероятности того, что модель выдала правильный ответ. Затем мы проверили температуру на втором наборе данных. Перед проверкой мы решили оставить температуру прежней только в том случае, если логарифмическая потеря на втором наборе данных снизится, а показатель Бриера не повысится. Оптимальная температура — 2,179. Хостинговый API и оба локальных счётчика используют её для этой версии.

При температуре 1 вероятность была слишком высока. Во втором наборе средняя вероятность выбранного ответа составила 0,89, но правильными оказались только 73% выбранных ответов. В таблице приведены результаты при обеих температурах. Чем ниже показатель, тем лучше.

Примеры	Измерять	Температура 1	Температура 2.179
Второй сет (300)	Ожидаемая ошибка калибровки	0.128	0.066
Второй сет (300)	Потеря журнала	0.692	0.555
Второй сет (300)	Оценка шиповника	0.348	0.295
Набор отложенных заявок (324)	Ожидаемая ошибка калибровки	0.052	0.054
Набор отложенных заявок (324)	Потеря журнала	0.318	0.259
Набор отложенных заявок (324)	Оценка шиповника	0.154	0.144

Для оценки ожидаемой ошибки калибровки мы сортируем примеры по десяти группам в зависимости от вероятности выбранного ответа. В каждой группе мы сравниваем среднюю вероятность с тем, насколько часто выбранный ответ оказывается верным. Ожидаемая ошибка калибровки — это среднее значение этих расхождений, взвешенное с учетом размера каждой группы. При оценке по шкале Брайера правильный ответ оценивается в 1 балл, а все остальные — в 0 баллов. Оценка по шкале Брайера — это сумма квадратов разностей между этими числами и вероятностями, усредненная по всем примерам.

В 24 из 300 примеров во втором наборе правильный ответ представляет собой набор вероятностей, а не один ответ. В этих примерах логарифмическая функция потерь и оценка Бриера сравнивают вероятности, полученные моделью, с эталонными вероятностями. При расчете ожидаемой ошибки калибровки эти 24 примера не учитываются. При обеих температурах модель выбрала правильный ответ в 220 из 300 примеров во втором наборе и в 292 из 324 примеров, не использованных для обучения.

Результаты не улучшились для всех типов вопросов. В 64 рейтинговых вопросах из 324 контрольных примеров вероятность выбранных ответов оказалась слишком низкой, учитывая, как часто эти ответы оказывались верными. Ожидаемая ошибка калибровки для этих вопросов выросла с 0,105 до 0,177.

Мы настроили температуру на пути CUDA, который загружает адаптер без объединения. В размещенном API, а также в руководстве по началу работы для Mac и Linux используются объединенные веса. Логиты от объединенных весов могут немного отличаться. Мы не проверяли температуру объединенных весов повторно.

🔗 Наблюдаемая задержка вывода

В приведенной ниже таблице указано время в миллисекундах для каждого примера. Мы рассчитали это время на основе данных с времени выполнения, которые мы сохранили для каждого запроса. В каждом примере один вопрос. Локальные модели и Jev возвращают типизированный ответ и вероятность каждого из возможных ответов. Модели OpenRouter генерируют текст обычным способом с настройкой аргументации на средний уровень. Для локальной оценки мы запускаем модель один раз для каждого примера. Мы не выполняем выборку более одного раза, и модель не генерирует объяснение.

Модель	Примеры	Медиана (мс)	Среднее значение (мс)	p95 (ms)	Run / dataset
Gemma 3 270M IT	324	21.8	23.6	29.2	H100 · contrastive holdout
Qwen3.5-0.8B	324	48.6	49.4	60.2	H100 · contrastive holdout
Qwen3.5-4B	324	58.0	58.6	70.3	H100 · contrastive holdout
Qwen3.5-9B	324	58.1	59.8	75.8	H100 · contrastive holdout
Qwen3.8-27B	324	145.3	145.2	185.5	H100 · contrastive holdout
Bespoke-Nimble-9B	120	106.0	110.1	119.8	H100 · contrastive holdout
Bespoke-Nimble-9B	324	444.0	546.0	981.0	M5 Pro 64GB · contrastive holdout
Jev 1.13.0	324	246.7	267.0	347.4	TypeSafe API · contrastive holdout
DeepSeek-V4.1-Flash	100	2896.4	5238.0	15065.4	OpenRouter / Fireworks · general eval
Qwen3.8 2.4T A95B	100	2792.3	3108.0	5110.5	OpenRouter / Modal · general eval

🔗 Документация и разработка

Я хочу...	Начните отсюда
Определение полей или понимание параллельного скоринга	Руководство по подсчету очков
Compare Nimble and Jev interactively	Comparison app
Host the published checkpoint on Modal	SGLang deployment · Try the public API
Understand the retained training and evaluation data	Dataset guide
Evaluate on public, human-labeled benchmarks	Public benchmarks
Create or replay contrastive training data	Curation guide
Train or use a schema adapter	Training guide

Compare two models only when both were tested on the same examples in the same way. The guides describe how to generate data and evaluation outputs locally; these generated files are not committed to Git.

🔗 Development

Use a separate Python environment for each of these, because they need different package versions:

- MLX
- PyTorch
- Curator

The dependency lists are in [requirements/](#). From the project root, run the offline tests that apply to your change.

```
.venv-mlx/bin/python -m unittest tests.test_parallel_scorer tests.test_evaluate_pilot tests.test_model_evaluation
.venv-curator/bin/python -m unittest tests.test_dataset_io tests.test_diverse_dataset
```

To measure how much time MLX saves by processing the shared context once for several fields, run the benchmark for schemas with several fields.

```
.venv-mlx/bin/python -m nimble.evaluation.benchmark_parallel --repeats 3
```

Each record in the saved dataset evaluations has only one field. So you cannot use those evaluations to measure the time saved by scoring several fields in parallel. Their recorded times are also not from a controlled serving test.

 Folder layout

```
nimble/
  scoring/      # Local MLX and PyTorch inference
  datasets/     # Generation, curation, and TypeSafe labeling
  evaluation/   # Quality metrics and inference benchmarks
  training/     # Schema-aware CUDA LoRA training
  examples/     # Schemas and saved example outputs
  data/         # train.jsonl (2,676), eval.jsonl (324), and verification metadata
  evaluations/  # Generated locally; not included in Git
  assets/       # Comparison graphics, source results, and videos
  docs/         # Current usage, training, curation, and deployment guides
  ignore/       # Local archive of historical docs and assets; not included in Git
  requirements/ # Backend-specific dependencies
  tests/        # Offline and small-model checks
```

 Citation

If you use this repository, please cite it. Also give the model revision and the dataset release that you used.

```
@misc{nimble2026,
  author = {{Bespoke Labs} and Sathiamoorthy, Maheswaran},
  title = {Nimble},
```

Releases

Packages

Used by

Contributors

Languages